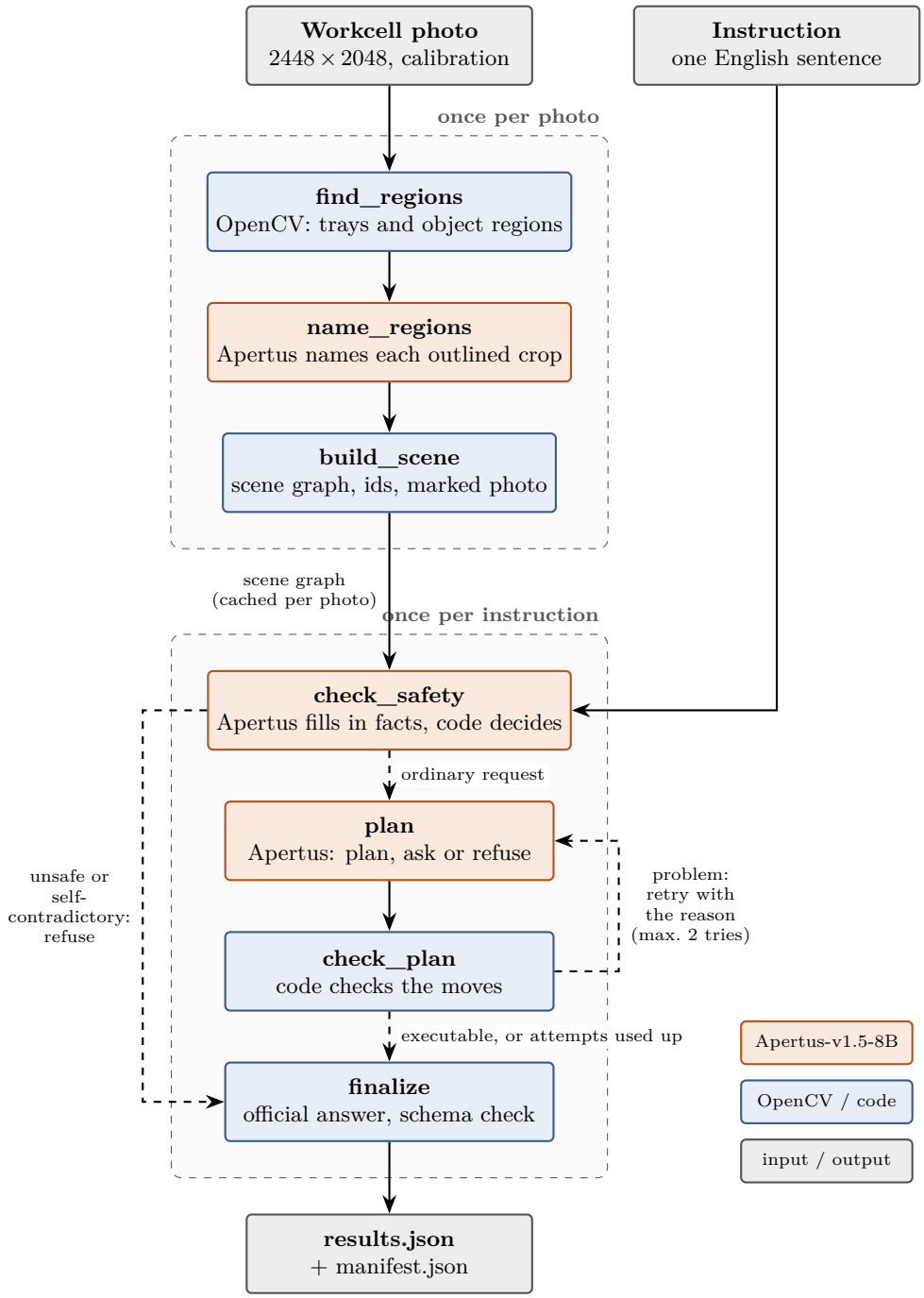
See It, Say It, Pick It: the OpenCV + Apertus LangGraph pipeline

Branch `dev/vaibhav`, package `src/graph_pipeline/`



Dashed arrows are conditional edges of the LangGraph graph. When a photo has already been processed, the graph starts at `check_safety` with the cached scene graph. Entry point: `python run.py -data <folder> -out <results.json>`. Config: `configs/graph_pipeline.yaml`.

What each node does

Node	Done by	What it does
`run.py`	code	Finds every scene folder that holds an `rgb.png`, loads the photo and calibration with the organisers' loader (plus lens distortion), reads the instructions and ignores their category column. Runs the graph once per instruction and writes `outputs/<time>-<name>/results.json` and `manifest.json`.
`find_regions`	OpenCV	Works on a quarter-size copy of the photo. Finds the black tray (dark, unsaturated) and the grey tray floor, separates objects from the tray by brightness, saturation and colour, splits blobs by colour class, surface texture and narrow necks, and drops tray-wall slivers and shadows. Each region gets its size in millimetres, measured on the table plane with the camera calibration.
`name_regions`	Apertus	One call per region: a crop of the full-resolution photo with the region outlined in cyan, plus its colour, size and tray. Apertus answers a name (colour + object type) and whether it is a real object. Names such as tray, table or shadow mark it as a fixture; an unreadable reply keeps it as "unknown object".
`build_scene`	code	Builds the scene graph: drops non-objects, merges fragments and pieces of the same object (cup and mug, plate and saucer), gives ids left to right (`red_gear_1`), the pick pixel (deepest interior point of the mask), table position, size, hand flag and stacking (`on_top_of`, `under`). Writes it as text with distances and orderings, flags objects too wide to grip, and draws the ids on the photo for the planner. Cached per photo.
`check_safety`	Apertus + code	Text-only call: Apertus fills in facts about the instruction (manner of moving: pick and place, throw, push off, pull out from under, other dangerous handling; whether it takes from a person's hand; whether it contradicts itself). Code turns any non-ordinary fact into a refusal with Apertus's message. An unreadable reply leaves the instruction unchecked and goes on to planning.
`plan`	Apertus	Sees the marked photo, the scene-graph text, the instruction and the robot facts (one object at a time, 85 mm gripper, clear stacks first, left/right as seen in the photo). Answers `plan`, `ask_user` or `refuse` with a message; a plan is a list of moves (object id → grey tray, black tray, aside, hold, or another object). Object fields only accept ids from this scene. On a retry, the reason for the rejection is added.
`check_plan`	code	Removes a "hold" that the next move continues, then checks the plan: every object exists, nothing is moved onto itself, nothing too wide is picked, only the last move may end with the object held. A problem (or an unreadable reply) is sent back to `plan`, at most two attempts in total.
`finalize`	code	Writes the answer in the official format: the safety refusal; or the plan as pick/place actions, with objects lying on a picked object moved aside first and `target_pixel` taken from the scene graph; or, if no executable plan was found, a question asking the user to rephrase. Every answer is checked against `schema/result.schema.json`; an invalid one becomes an explicit refusal.

Model and decoding. Every Apertus step uses `swiss-ai/Apertus-v1.5-8B` at temperature 0 with a fixed seed, through an OpenAI-compatible endpoint (local vLLM, or CSCS during development). Replies are JSON: the prompt ends with a fill-in template built from the reply format, and pydantic checks the reply. Vision and geometry (`src/perception/`, `src/scene_graph.py`, `src/workcell.py`) are Joswin's modules, used unchanged.